Microservices identification strategies

A review focused on Model-Driven Engineering and Domain Driven Design approaches

Roger Anderson Schmidt
Laboratory in Software Quality and Productivity – LQPS
University of Vale do Itajaí – UNIVALI
Florianópolis, Brazil
rasflp@gmail.com

Marcello Thiry
Laboratory in Software Quality and Productivity – LQPS
University of Vale do Itajaí – UNIVALI
Florianópolis, Brazil
marcello.thiry@gmail.com

Abstract — **A proper architectural design for a microservices system is crucial for its success. Although there are several design strategies to identify software components in general, microservices demands special consideration. In this context of distributed systems, the component size directly impacts on non-functional requirements, such as performance, flexibility, reusability, etc. Design practices of coupling and cohesion have to be fine-tuned to determine the ideal microservices granularity. In order to shed light on this question, this study conducted a Systematic Literature Review that investigates microservices identification proposals. From procedures and guidelines inspired by Kitchenham et al., a rigorous research protocol was defined and performed, that covers publications from 2013 to 2019. Starting with an initial screening of 715 papers, 27 studies were considered relevant to answer four research questions. Besides microservices decomposition strategies, this review underlines Model Driven Engineering and Domain Driven Design, once they represent valuable approaches to support this challenging task. Moreover, this work highlights that only a few studies had explored these approaches in their strategies, which opens promising potential for further research.**

Keywords – microservices, identification, decomposition, granularity, MDE, DDD, model-driven, domain-driven.

I. INTRODUCTION

The academic background that offers scientific support to microservices development has increased in the last few years. However, some aspects of software design specifically related to this architectural view demand additional investigation [1], [2]. In this research context, finding the optimal granularity for these small services represents a challenging task. Breaking down an application domain into properly sized components as a microservice is not an easy task because the notion of the ideal size for these components seems blurred or unclear [3], [4].

Among questions behind the microservice, the functional decomposition of the business domain into services represents the essence of this architectural pattern [4]. Although a decomposition from a business perspective is valid, there are some obstacles that have to be considered, for instance, network latency, limited availability for synchronous communication, maintaining data consistency across services, obtaining a consistent view of the data and classes that promote excessive dependency [4].

Moreover, in addition to the introduced challenge of microservices optimal granularity, this research dedicates special interest to the application of two valuable techniques, however, still academically under-explored: Model-Driven Engineering and Domain-Driven Design. In agreement with the idea that services are more properly organized around business rather than technical concerns, the Domain-Driven Design (DDD) approach can be conveniently applied. Towards this goal, DDD offers principles, practices and patterns, as guidelines for the application domain decomposition, from a business context view [3], [4]. Besides, Model-Driven Engineering (MDE) is another approach of special interest in the present study. Its capability to create model instances from metamodels and performing transformations opens possibilities for developing resources to support the microservices granularity definition.

In this context, the present study conducts a Systematic Literature Review (SLR) that aims to understand, evaluate and compare proposals, not only for microservices identification in new applications, but also for migrations from legacy systems, or monoliths. This SLR provides a background literature that could inspire potential proposals based on the combination of DDD and MDE approaches to address the design challenge introduced in this section. Even though interest in microservices has been increasingly growing, research on its architectural design in terms of size, orchestration and integration remains under-explored [1]. Similarly, the usage of DDD and MDE in the context of microservices, individually or combined, could open interesting possibilities for research that justify more investigation in the academia.

The remainder of this article is structured as follows. Section II presents the methodological aspects which established the scientific bases for this systematic literature review. Then, the results obtained from data extraction and synthesis of the selected papers are discussed in Section III. Finally, the last sections present our strategies to minimize threats to validity, conclusions produced from outcomes of this review, and our plans to apply the contributions provided from this investigation towards the development of related research in the future.

II. METHODOLOGY

This research is based on Kitchenham et al. [5], [6] methodology, which compiles procedures and guidelines specifically related to SLRs in the Software Engineering field. In accordance with the first stage of the planning phase, that is required to ensure that a new systematic review is really necessary, by applying the review protocol in order to examine

2020 15th Iberian Conference on Information Systems and Technologies (CISTI)
24 – 27 June 2020, Seville, Spain
ISBN: 978-989-54659-0-3

existing secondary studies. Therefore, the relevant SLRs were selected, evaluated and interpreted, as concisely presented in the following section.

A. The need for a systematic review

As a result of the execution of our research literature protocol in order to find related SLRs, we found publications around a similar context, but wider in scope. The selection process started with an initial search through four databases presented in Table 4, which returned 22 papers. The applied criteria are presented in sections D and E. Besides, the terms related to secondary studies (SLRs) were added to the base search string (Section F) After metadata filtering, 16 studies were dismissed, so the 6 remaining studies were selected for detailed filtering by a thorough paper reading. As a result, 2 secondary studies were selected, which were evaluated as follows.

Francesco et al. [7] offers a comprehensive systematic review on microservices architectural view. Although this work covered 19 studies that researched the related fields, not all of these papers develop proposals for application domain decomposition into microservices. Therefore, some of the studies examined in the related reviews are exclusively concerned with performance, scalability and other non-functional requirements.

Andriyanto and Doss [8] explores aspects of service oriented architecture, mostly related to the traditional concept of SOA. It also covers microservices as an extension, evolution or subset of SOA, however its scope goes beyond granularity issues, so it does not perform an in-depth investigation into this phenomenon of interest.

On the other hand, our review seems to be more specific than other related studies, as the research problem explored consists in finding proposals for identification, decomposition, partitioning or breaking down the application domain in order to reach an adequate granularity for microservices. Moreover, the research aims to highlight the usage of MDE or DDD approaches in this context.

B. Review protocol

In accordance to Kitchenham et al. guidelines [9], SLRs start by defining a protocol that guides the research process, which specifies central questions and procedures that are used to conduct the review. The following sections briefly present the protocol that guided this SLR, which consists of two consecutive workflows, as shown in Figure 1.

C. Research questions

Considering the central importance of research questions, the fact that they drive the entire methodology, the PICO(C) [10] model was applied to assess their identification process. By applying it, we defined five perspectives, as shown in Table 1, that made it clear to compose questions inside the research scope. For the sake of conciseness, these four questions are presented with the answers in Section III.

D. Inclusion criteria

- Publications available at these 4 online databases: Elsevier SCOPUS, ACM Guide to Computing Literature, IEEE Xplore Digital Library, and ScienceDirect (Elsevier)
- Papers published from 01/01/2013 to 31/12/2019
- Articles published in journals, conferences and workshops (with peer review)
- Studies that propose microservices identification strategies or that apply MDE techniques in this context.

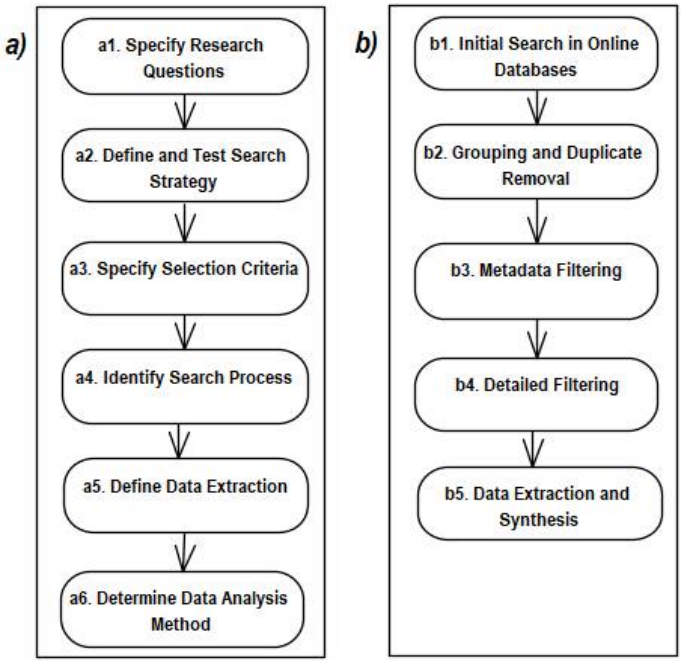


Figure 1. Review protocol development (a) and primary studies selection (b) workflows

E. Exclusion criteria

- Studies not written in English
- Publications without abstract
- Books, websites, technical reports and academic thesis

TABLE 1. PERSPECTIVES IDENTIFIED FROM PICO (C) MODEL

Criteria	Value
Population	System analysts, designers and domain experts
Intervention	Strategies for application domain decomposition into microservices, by highlighting two supporting approaches, in special: DDD and MDE
Comparison	Evaluation of distinct strategies through the application of a defined criteria
Results	Approaches, methods or techniques for microservices granularity definition
Context	Design and modeling activities related to the application domain decomposition, along a development process phase, in which functional microservices are identified

- Studies focused on microservices deployment, for instance, those exclusively related to benchmarking, containerization, and parallel aspects.

F. Search String Composition

Considering the restriction for papers only published in English, the keyword "microservice" is the main term, with mandatory presence in papers metadata. Besides that, it is desirable to recover papers that explore model driven approaches, which can be found in the literature as: Model-Driven Engineering (MDE), Model-Driven Development (MDD), Model-Driven Software Engineering (MDSE), and others. Thus, the addition of "model" or "model-driven" to the search string is intended to contemplate all variations. Still, according to the research theme, the term "domain" is relevant, in order to recover approaches notably explored in this context, like Domain-Driven Design (DDD). Finally, in relation to the application domain, this review is centered on the question of how to proceed with decomposition, partitioning, granularity definition or breaking down into microservices. As a result of several test iterations, the search string is essentially composed from the terms presented in Table 2.

TABLE 2. SEARCH STRING COMPOSITION

Term	Acronyms and synonyms
P = Microservice (Architecture and component itself)	"microservice*"
Q = Model-Driven Software Engineering, or acronyms	"model*", "mde", "mdse"
S = Domain-Driven Design, Domain (in general), or acronym	"domain", "ddd"
T = (De)composition or Partitioning or Granularity or Breaking Down	"*compos*", "partition*", "granular*", "break*"
Expression	P ∩ (Q ∪ S ∪ T)

G. Study selection

As presented in Figure 1 (b), the selection process for publications is composed of 5 activities. Table 3 shows the numbers of papers per database, throughout the stages of this process. First, the search expression is submitted to each corresponding database. Then its entries are exported from each database portal to a BibTeX format, and the whole paper metadata (title, abstract and keywords) are cataloged in the Mendeley tool, which made it easy to find and remove 95 initial duplicates from 810 entries to start the selection process.

Following the referred workflow, we evaluated the metadata from each article in terms of the literature selection criteria (b3). After dismissing 3 incomplete entries, 137 articles were selected for detailed filtering. As a result of full reading (b4), 27 papers were selected for data extraction and analysis (b5). Table 3 shows the amount of studies throughout the selection process.

TABLE 3. AMOUNT OF PRIMARY STUDIES ALONG THE SELECTION PROCESS

Database	Initial Search (b1)	Duplic. Removal (b2)	Metadata Filtering (b3)	Detailed Filtering (b4)	Final Selection (b5)
Scopus	248	95	137	123	27
ACM DL	79				
IEEE Explore	232				
ScienceDirect	260				
Total	810	95	137	123	27

III. RESULTS

Once the selection process was finished, 27 primary studies referenced in Table 5 were thoroughly examined for data extraction and synthesis in order to clarify the four research questions. All studies are identified by the letter "S" followed by its sequence. Considering the origin of the papers, Germany and China are the most representative countries, and, regarding the type of publication, the vast majority of publications were presented at international conferences.

Although the time frame adopted in inclusion criteria is wider, the selected studies range from 2016 to 2019. Next, we discuss our findings about each research question, which is the main contribution of this research.

RQ1: Which strategies, techniques or methods have been adopted for domain decomposition into microservices and how are they applied to stablish its optimal granularity?

The study S1 applies Requirements Engineering to propose a method that produces a set of microservices candidates, which is based on metrics for security and scalability calculated from functional requirements. Clustering techniques are applied on business activities by the study S3 to identify services from process models expressed in BPMN. Continuing with clustering techniques, the Affinity Propagation algorithm is also explored by S18 which takes OpenAPI specifications as an input, in order to identify functions that are related to each other based on semantic similarity of operation names. S23 also explores Natural Language Processing (NLP) over Behavior Driven Development (BDD) scenarios in the context of smart-city applications. Clustering is also applied in S27 to identify URIs with similar performance and resource requirements by using historical access logs.

Dynamic models are explored in S4 and S22 also, by applying transformations over Dataflow Diagrams (DFDs). On the other hand, by following a static model, S6 proposes an approach driven by the application data model, which considers the frequency of calls between services derived from requirements. The article S17 explores role-based modeling techniques, which apply Role Modeling (RM) representations. Both studies, S17 and S20, apply ontologies in their proposals. The latter translates profiles in EBNF to ontologies and performs a matching process between properties of decomposed functions of components in order to fully or partially replace their functional part with microservices.

The study S25 proposes a 6-step process-mining approach, which provides a reasoning framework to assess the software architect finding the best decomposition solution according to a metric-based evaluation. S21 addresses the cost impact in granularity decisions in the application of microservices for Function Blocks (FB) architecture from Industrial Automation Standards, by developing a mathematical method that evaluates

times elapsed for service de/composition considering related process complexity and overhead.

The following studies apply Autonomic Computing (AC) techniques to design their decomposition methods. The study S9 explores a self-adaptive mechanism focused on microservices granularity aspect and based on a MAPE-K loop implementation. The study S12 uses a genetic algorithm to suggest microservices candidates in terms of workload monitored at runtime, by focusing on systems performance and scalability. Similarly, a genetic algorithm is also applied in the proposal S3, by considering two aspects of dependency: structural and of data object.

TABLE 4. SELECTED PRIMARY STUDIES (RQ1)

Study N. [Ref.]	Relevant Inputs	Autom. Level
S1 [11]	Behavioral (functional requirements or use cases), corresponding metrics for RNF	Semi
S2 [12]	Static (structural components from source code) and Dynamic (data collecting at runtime)	Full
S3 [13]	Nodes and edges from a process modeled BPMN	Semi
S4 [14]	Purified DFD Diagram from a business scenario	Semi
S5 [15]	Instance of Ecore Metamodel	Full
S6 [1]	ER Model and business services	None
S7 [16]	Costs of quality assurance, plus automated and non-automated deployment	None
S8 [17]	Static (structural components from source code) and Dynamic (data collecting at runtime)	Full
S9 [18]	System at runtime, ambients architectural configuration	Full
S10 [19]	Instrumented system and test cases	Full
S11 [20]	Domain knowledge	None
S12 [21]	Feature Model, microservices system	Semi
S13 [22]	Domain Model	Full
S14 [2]	Domain Model	Semi
S15 [23]	Microservices system at runtime	None
S16 [24]	AjiML Diagrams of Services (in concrete syntax)	Full
S17 [25]	Role Model Ontology, identification and categorization of services	None
S18 [26]	OpenAPI specifications of MSA projects	Full
S19 [27]	Logfiles and UML Models	Full
S20 [28]	Semi-structured form of natural language (EBNF)	Full
S21 [29]	Services requests routes under monitoring	Semi
S22 [30]	DFD Diagram	Semi
S23 [31]	Microservices projects and requirement scenarios written in BDD form	Semi
S24 [32]	Model expressed in the language proposed and corresponding aspects implemented	None
S25 [33]	Execution traces of the system	Full
S26 [34]	Domain knowledge	None
S27 [35]	Historical logs with URI requests from a monolithic web application	Full

In relation to non-functional requirements addressed by the proposals, Figure 2 shows a distribution chart among the studies. Design and modeling features figures as the most frequent requirements, since they are directly related to the phenomena investigated by this SRL. Performance is the focus of 6 studies for benchmarking or monitoring at runtime. In similar direction, S24 employs Aspect Oriented Modeling (AOM) to implement an abstraction layer called "view-point" to support technology heterogeneity in microservices projects.

Other studies propose their identification strategies from DDD principles and patterns (S7, S11, S13, S14, S26), that are specifically addressed by our fourth research question. Some studies don't develop domain decomposition strategies directly; however, they offer verification platforms of performance, resilience and latency (S15, S19). The S19 proposal connects design and runtime, which stores distributed tracing information for messages being sent to endpoints representing the microservices of an application in order to identify performance antipatterns.

All proposals presented in S2, S3 and S8 apply Software Architecture Recovery (SAR) to recover architectural information from projects available in code repositories. In this group, study S16 offers an MDE tool for modeling and code generation, which is particularly oriented towards systems of microservices. Through system traces evaluation collected at runtime, study S10 also suggests potential microservices.

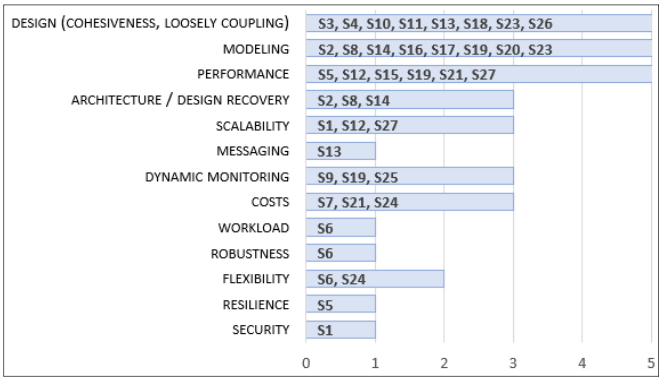


Figure 2. Non-functional requirements addressed

In order to provide additional information about the workflows, Table 4 presents what we considered the most relevant inputs for each proposal. The automation level represents our understanding in terms of how straightforward the decomposition strategy seems, ranging from None, Semi or Full.

RQ2: Which roles are involved in the proposed microservices identification process and what are their main responsibilities?

This question is intended to provide some background about the level of abstraction and the set of skills that is needed to conduct each service identification proposal, among the roles of a non-specific software development process. Although the recommended role of the process is only explicitly assigned in 6 studies (S1, S4, S8, S13, S22, S26), many roles can be inferred

by context. Other proposals require Software Engineering skills (S15, S10, S7, S9, S4, S19, S20, S22). Among these studies, S4 and S22 require final user engagement, as well. The Software Architect is identified as an active role in most of the proposals (S2, S5, S8, S13, S15, S10, S18, S23-S25, S27). Despite explicitly mentioned, it is understood that the Software Designer would be the role responsible for elaborating a domain model presented in studies S16 and S17.

Business Analyst or Domain Expert can be assigned as responsible to conduct the technique in S3, S6, S11, S12, S14 and S26, as the last five also allocate other roles, in general. There is some uncertainty over the identification of the roles assigned to the activity proposed by S7, since it is related to cost related aspects. The proposal S22 explicitly assigns Requirement Analyst, Data Analyst and Software Architect in its dataflow-driven decomposition mechanism. The study S1 assigns Requirements Engineer as the main responsible for applying its methodology. As expected, most of the role's responsibilities are modelling activities, by producing representations of the corresponding models. The level of abstraction for each model varies in relation to the automation level of each proposal.

RQ3: Do the selected proposals apply Model-Driven Engineering (MDE) and which related elements have been explored?

From the 27 selected studies, 10 actually explore MDE, in terms of transformations in a model-driven process. A list of all representation types produced is shown in Table 5. Among MDE techniques explored by the selected studies, metamodeling is the most frequent (S2, S5, S8, S9, S19, S20, S23, S24). Besides that, the study S9 applies Architecture Description Language (ADL), and Aspect Oriented Programming (AOP), which is explored by S24 for modeling (AOM), as well. The study S5 presents an MDE process, accompanied by the development of an Ecore metamodel and an artifact generator built from XTend framework templates. The study S13 implements the transformation of models expressed as UML Component Diagrams, by following the proposal of Model-Driven Architecture (MDA), in which the DDD domain model corresponds to the Platform Independent Model and the microservices model appears as the Platform Specific Model.

The S14 offers an UML profile to provide support for representing a domain model as UML Class Diagram. The S16 explores two MDE techniques: Domain-Specific Modeling Language (DSML), which is built upon the Sirius framework, and Model-To-Code (M2C) transformations, by applying Acceleo and Eclipse frameworks in order to produce structures based on the Java language and Spring Cloud Netflix framework. The S19 uses JTL (Janus Transformation Language) to define correspondences between the system behavior and the architectural design. In a similar way, the S20 implements C2M (Code-to-Model) transformations, using ANTRL framework to build ontology tree instances described by EBNF profiles. Study S23 generates a "Service Dependency Graph (SDG)" to visualize microservice relationships.

The S24 implements a language to declare technological aspects of microservices implementation, specified in Xcore language and implemented with Xtext framework. This approach creates an independent layer aligned to the concept of Platform Independent Model (PIM) from MDA.

TABLE 5. REPRESENTATIONS PRODUCED BY THE PROPOSALS (RQ3)

Representation Type	Studies
Use Case Diagram	S1
Class Diagram	S2, S17, S19, S24
BPMN Model	S3
Dataflow Diagrams (DFD)	S4, S22
Ecore model	S5, S19
ER Model	S6
Domain Model	S13, S14
Deployment-based Diagram	S15, S19
Ontology Model	S17, S20
Non-standardized representations	S7-S12, S16, S21, S23, S25, S27

Despite the fact that studies S3, S4/S22, S6 and S21 adopt abstractions in terms of models expressed in BPMN, DFD and MER, respectively, no MDE process is conducted.

RQ4: Do the selected studies cover Domain-Driven Design (DDD) at any level and in what way are its principles, practices and patterns applied?

Only 4 studies include DDD techniques in their proposals, as listed in Table 6. Bounded context is the most recurring DDD concept among the selected studies. The S7 briefly mentions the application of DDD, so it doesn't provide more detail about the process. S23 emphasizes DDD in its microservices retrieval method, pointing out the large number of words used to describe domains and scenario-driven applications. The S11 presents a methodology for domain decomposition by applying DDD in a systematic way. The studies S13 and S14 guide the adoption of DDD patterns, placing them inside microservices architecture context.

TABLE 6. DDD APPLIED TO THE PROPOSALS (RQ4)

SID	Proposal Core
S7	Concept of Bounded Context (BC) applied for monolith decomposition into microservices
S11	Systematic procedure for DDD application to microservices systems partitioning
S13	DDD Context Mapping and BCs, in which communication between microservices is modeled as publish-subscribe-channel EIP messaging pattern
S14	Class Diagram with elements stereotypes as DDD patterns and elements
S26	Metamodel of the CSD domain

IV. CONCLUSIONS

This review examined 27 strategies on microservices identification, which were extracted and analyzed through a systematic process, from an initial selection of 715 primary studies. Resolution time is an important aspect to observe from the proposed methods. Some proposals collect data at runtime,

generally using a tracing system, by evaluating aspects of its dynamic behavior. Another group of studies suggest microservices from business models, such as requirements models, business processes and data flows. Still, some studies explore static elements in models to design microservices based on entities or roles.

As a future work, this review will build the literary foundation for a microservices identification proposal based on business capabilities with DDD. In this project, we aim to design a model-driven process composed by different levels of abstraction, from domain model to source-code.

REFERENCES

[1] B. Götz, D. Schel, D. Bauer, C. Henkel, P. Einberger, and T. Bauernhansl, "Challenges of Production Microservices," Procedia CIRP, vol. 67, pp. 167–172, 2018.

[2] F. Rademacher, J. Sorgalla, and S. Sachweh, "Challenges of domain-driven microservice design: A model-driven perspective," IEEE Softw., vol. 35, no. 3, pp. 36–43, 2018.

[3] S. Newman, Building Microservices. Sebastopol, CA: O'Reilly Media, Inc., 2015.

[4] C. Richardson, Microservices Patterns. New York, NY: Manning Publications Co, 2018.

[5] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering - A systematic literature review," Inf. Softw. Technol., vol. 51, no. 1, pp. 7–15, 2009.

[6] B. Kitchenham et al., "Systematic literature reviews in software engineering – A tertiary study," Inf. Softw. Technol., vol. 52, no. 8, pp. 792–805, 2010.

[7] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," J. Syst. Softw., vol. 150, pp. 77–97, 2019.

[8] A. Andriyanto and R. Doss, "A Literature Review of Inter-enterprise SOA in Small and Medium Business Communities," pp. 547–554, 2018.

[9] B. Kitchenham, D. Budgen, and P. Brereton, "Guidelines for performing systematic literature reviews in software engineering," Int. Conf. Soft. Engin., vol. 45, no. 4ve, p. 1051, 2006.

[10] M. Petticrew and H. Roberts, Systematic Reviews in the Social Sciences: A Practical Guide. 2006.

[11] M. Ahmadvand and A. Ibrahim, "Requirements Reconciliation for Scalable and Secure Microservice (De)composition," in 2016 IEEE 24th International Requirements Engineering Conference Workshops (REW), 2016, pp. 68–73.

[12] N. Alshuqayran, N. Ali, and R. Evans, "Towards Micro Service Architecture Recovery: An Empirical Study," in 2018 IEEE International Conference on Software Architecture (ICSA), 2018, pp. 47–4709.

[13] M. J. Amiri, "Object-Aware Identification of Microservices," in 2018 IEEE Intl Conf. on Services Computing (SCC), 2018, pp. 253–256.

[14] R. Chen, S. Li, and Z. Li, "From Monolith to Microservices: A Dataflow-Driven Approach," in 2017 24th Asia-Pacific Software Engineering Conference (APSEC), 2017, pp. 466–475.

[15] T. F. Düllmann and A. Van Hoorn, "Model-driven generation of microservice architectures for benchmarking performance and resilience engineering approaches," in ICPE 2017 - Companion of the 2017 ACM/SPEC International Conference on Performance Engineering, 2017, pp. 171–172.

[16] J.-P. Gouigoux and D. Tamzalit, "From monolith to microservices: Lessons learned on an industrial migration to a web oriented architecture," in Proceedings - 2017 IEEE Intl Conf. on Software Architecture Workshops, ICSAW 2017: Side Track Proceedings, 2017, pp. 62–65.

[17] G. Granchelli, M. Cardarelli, P. D. Francesco, I. Malavolta, L. Iovino, and A. D. Salle, "Towards recovering the software architecture of microservice-based systems," in Proceedings - 2017 IEEE International Conference on Software Architecture Workshops, ICSAW 2017: Side Track Proceedings, 2017, pp. 46–53.

[18] S. Hassan, N. Ali, and R. Bahsoon, "Microservice Ambients: An Architectural Meta-Modelling Approach for Microservice Granularity," in Proceedings - 2017 IEEE Intl Conf on Software Architecture, ICSA 2017, 2017, pp. 1–10.

[19] W. Jin, T. Liu, Q. Zheng, D. Cui, and Y. Cai, "Functionality-Oriented Microservice Extraction Based on Execution Trace Clustering," in 2018 IEEE Intl Conf on Web Services (ICWS), 2018, pp. 211–218.

[20] M. I. Josélyne, D. Tuheirwe-Mukasa, B. Kanagwa, and J. Balikuddembe, "Partitioning microservices: A domain engineering approach," in Proceedings - International Conference on Software Engineering, 2018, pp. 43–49.

[21] S. Klock, J. M. E. M. Van Der Werf, J. P. Guelen, and S. Jansen, "Workload-Based Clustering of Coherent Feature Sets in Microservice Architectures," in Proceedings - 2017 IEEE International Conference on Software Architecture, ICSA 2017, 2017, pp. 11–20.

[22] R. Petrasch, "Model-based engineering for microservice architectures using Enterprise Integration Patterns for inter-service communication," in Proceedings of the 2017 14th International Joint Conference on Computer Science and Software Engineering, JCSSE 2017, 2017.

[23] D. Shadija, M. Rezai, and R. Hill, "Microservices: Granularity vs. Performance," in Companion Proceedings of the10th International Conference on Utility and Cloud Computing, 2017, pp. 215–220.

[24] J. Sorgaila, P. Wizenty, F. Rademacher, S. Sachweh, and A. Zündorf, "AjiL: Enabling Model-driven Microservice Development," in Proceedings of the 12th European Conference on Software Architecture: Companion Proceedings, 2018, pp. 1:1--1:4.

[25] P. Tsoutsa, P. Fitsilis, and O. Ragos, "Role modeling of IoT services in industry domains," in ACM International Conference Proceeding Series, 2017, pp. 290–295.

[26] O. Al-Debagy and P. Martinek, "A new decomposition method for designing microservices," Period. Polytech. Electr. Eng. Comput. Sci., vol. 63, no. 4, pp. 274–281, 2019.

[27] D. Arcelli, V. Cortellessa, D. Di Pompeo, R. Eramo, and M. Tucci, "Exploiting architecture/runtime model-driven traceability for performance improvement," in Proceedings - 2019 IEEE International Conference on Software Architecture, ICSA 2019, 2019, pp. 81–90.

[28] A. Christoforou, L. Odysseos, and A. S. Andreou, "Migration of software components to microservices: Matching and synthesis," in ENASE 2019 - Proceedings of the 14th International Conference on Evaluation of Novel Approaches to Software Engineering, 2019, pp. 134–146.

[29] A. Homay, A. Zoitl, M. De Sousa, M. Wollschlaeger, and C. Chrysoulas, "Granularity cost analysis for function block as a service," in IEEE International Conference on Industrial Informatics (INDIN), 2019, vol. 2019-July, pp. 1199–1204.

[30] S. Li et al., "A dataflow-driven approach to identifying microservices from monolithic applications," J. Syst. Softw., vol. 157, 2019.

[31] S.-P. Ma, C.-Y. Fan, Y. Chuang, I.-H. Liu, and C.-W. Lan, "Graph-based and scenario-driven microservice analysis, retrieval, and testing," Futur. Gener. Comput. Syst., vol. 100, pp. 724–735, 2019.

[32] F. Rademacher, S. Sachweh, and A. Zundorf, "Aspect-oriented modeling of technology heterogeneity in microservice architecture," in Proceedings - 2019 IEEE International Conference on Software Architecture, ICSA 2019, 2019, pp. 21–30.

[33] D. Taibi and K. Systä, "From monolithic systems to microservices: A decomposition framework based on process mining," in CLOSER 2019 - Proceedings of the 9th International Conference on Cloud Computing and Services Science, 2019, pp. 153–164.

[34] C. Zaschke, "Elaboration of a domain model for migrating the monolithic software architecture of a data management server into a microservice architecture," in IC3K 2019 - Proceedings of the 11th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management, 2019, vol. 3, pp. 212–218.

[35] M. Abdullah, W. Iqbal, and A. Erradi, "Unsupervised learning approach for web application auto-decomposition into microservices," J. Syst. Softw., vol. 151, pp. 243–257, 2019.

Copyright of CISTI (Iberian Conference on Information Systems & Technologies / Conferência Ibérica de Sistemas e Tecnologias de Informação) Proceedings is the property of Conferencia Iberica de Sistemas Tecnologia de Informacao and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.